

# ONEDRAFT

## CAD-AI — Aria Powered

### Testing, Quality Assurance & Verification System

#### Version 1.0

**Status:** Implementation Specification

**System:** OneDraft CAD-AI Platform

**Subsystem:** Testing, Quality Assurance & Verification

**Primary Objective:** Establish the automated and deterministic mechanisms by which OneDraft verifies the correctness, integrity, reproducibility, safety, and operational reliability of its software, computational project model, geometry, compliance analysis, documentation, runtime services, and generated deliverables.

---

## 1. TESTING SYSTEM PRINCIPLE

Testing is a first-class subsystem of OneDraft.

OneDraft shall not treat testing as a final development activity performed after implementation.

Testing shall exist continuously across:

**source code**

**domain services**

**database**

**API**

**geometry**

**parametric relationships**

**compliance**

**validation**

**documentation**

**AI commands**

**runtime orchestration**

**workers**

**artifacts**

**security**

**integrations**

**user workflows**

and:

**final project acceptance**

The testing system therefore becomes an independent verification layer over the OneDraft computational system.

The objective is not merely to determine whether software executes.

The objective is to determine whether the system produces the **correct project state from defined inputs while preserving constraints, provenance, version integrity, regulatory evidence, and reproducibility.**

---

## 2. QUALITY ASSURANCE PRINCIPLE

OneDraft shall distinguish between:

### **Testing**

determining whether a defined behavior produces the expected result.

### **Verification**

determining whether the implementation satisfies its technical specification.

### **Validation**

determining whether the resulting project state satisfies defined requirements, constraints, rules, and acceptance criteria.

### **Quality assurance**

establishing the processes and controls that make those outcomes repeatable.

The four concepts shall remain related but distinct.

---

## 3. TESTING ARCHITECTURE

The testing architecture shall operate across the following layers:

SOURCE

↓

UNIT TESTS  
↓  
DOMAIN TESTS  
↓  
DATABASE TESTS  
↓  
API / CONTRACT TESTS  
↓  
INTEGRATION TESTS  
↓  
GEOMETRY TESTS  
↓  
COMPLIANCE TESTS  
↓  
DOCUMENTATION TESTS  
↓  
AI COMMAND TESTS  
↓  
END-TO-END TESTS  
↓  
REGRESSION TESTS  
↓  
ACCEPTANCE TESTS  
↓  
RELEASE VERIFICATION

No individual testing layer replaces another.

---

## 4. TEST ENVIRONMENTS

The platform shall maintain separated environments:

LOCAL  
DEVELOPMENT  
TEST  
STAGING  
PRODUCTION

Production data shall not be used as unrestricted development test data.

Test environments shall use controlled configuration and identifiable test datasets.

---

## 5. TEST DATA PRINCIPLE

Test data shall be deterministic wherever practical.

A test dataset shall identify:

dataset\_id  
version  
source

created\_at  
content\_hash  
schema\_version  
purpose

Representative datasets shall include:

**residential projects**

**commercial projects**

**multi-level buildings**

**irregular geometry**

**complex wall assemblies**

**accessibility conditions**

**fire and life-safety conditions**

**site constraints**

**multi-building projects**

**large documentation packages**

**redline revisions**

and:

**regulatory conflicts**

---

## 6. TEST FIXTURES

Reusable fixtures shall be established for:

organization  
user  
project  
site  
building  
level  
space  
element  
assembly  
constraint  
relationship  
revision  
model version  
code manifest  
validation run  
drawing  
sheet  
redline

document package  
acceptance record

Fixtures shall be version controlled.

---

## 7. UNIT TESTING

Unit tests shall verify individual functions and classes in isolation.

Initial unit-test targets include:

domain models  
validators  
geometry utilities  
constraint evaluators  
unit conversion  
identifier generation  
state transitions  
hash generation  
command parsing  
authorization checks  
pagination  
serialization  
deserialization

A unit test shall have:

input  
expected behavior  
expected output  
assertion

---

## 8. DOMAIN SERVICE TESTING

Domain services shall be tested independently of external infrastructure where practical.

Examples include:

ProjectService  
ModelService  
RevisionService  
ConstraintService  
CommandService  
ComplianceService  
ValidationService  
DrawingService  
RedlineService  
AcceptanceService

Tests shall verify business invariants rather than implementation details alone.

---

## 9. DATABASE TESTING

Database tests shall verify:

**schema creation**

**migration ordering**

**foreign keys**

**unique constraints**

**indexes**

**transaction behavior**

**rollback behavior**

**soft deletion**

**historical preservation**

**revision integrity**

**referential integrity**

**concurrent access behavior**

The database test suite shall execute against an actual PostgreSQL test instance rather than relying exclusively on mocks.

---

## 10. MIGRATION TESTING

Every migration shall be tested in both directions where rollback is supported.

The test process shall verify:

```
empty database
↓
migration 001
↓
migration 002
↓
...
↓
current schema
```

A clean environment must be capable of reproducing the current database schema from the migration chain.

---

# 11. TRANSACTION TESTING

Critical model mutations shall be tested for atomicity.

Example:

```
COMMAND
↓
VERIFY
↓
MUTATE
↓
RECORD CHANGE
↓
CREATE REVISION
↓
EVENT
↓
COMMIT
```

If any required stage fails:

ROLLBACK

The test shall confirm that no partial project state remains.

---

# 12. API TESTING

Every externally exposed endpoint shall have automated tests covering:

**valid requests**

**invalid requests**

**authentication**

**authorization**

**validation errors**

**missing resources**

**conflicting revisions**

**duplicate requests**

**idempotency**

**pagination**

**filtering**

**serialization**

## 13. API CONTRACT TESTING

The OpenAPI specification shall be treated as an executable contract.

Tests shall verify that:

request schema  
response schema  
status code  
headers  
error structure  
authentication requirements

conform to the OpenAPI definition.

The implementation shall not silently diverge from:

/docs/openapi/openapi.yaml

---

## 14. IDEMPOTENCY TESTING

Mutating requests supporting:

Idempotency-Key

shall be tested by submitting the same request multiple times.

Expected behavior:

REQUEST 1

↓

EXECUTE

REQUEST 2

↓

RECOGNIZE DUPLICATE

↓

RETURN ORIGINAL RESULT

No duplicate project mutation shall occur.

---

## 15. CONCURRENCY TESTING

OneDraft shall test concurrent operations against the same project.



Examples:

two simultaneous revisions  
two Aria commands  
simultaneous redline updates  
drawing generation during model modification  
validation during revision creation

The system shall prevent stale state from becoming authoritative.

---

## 16. OPTIMISTIC CONCURRENCY TESTING

Commands containing:

expected\_revision

shall fail when the expected revision is no longer current.

Example:

CLIENT  
expects Revision 14

SERVER  
currently Revision 15

RESULT  
CONFLICT

The test shall confirm that the command does not modify the project.

---

## 17. MODEL INVARIANT TESTING

The Project Model shall have explicit invariants.

Examples:

**levels belong to buildings**

**spaces belong to levels**

**elements may reference valid hosts**

**relationships reference valid objects**

**revisions belong to projects**

**model versions belong to projects**

**drawings reference valid views**

**document packages reference valid revisions**

**acceptance records reference valid document packages**

Tests shall verify these invariants continuously.

---

## 18. GEOMETRY TESTING

Geometry is a computational subsystem and requires specialized testing.

Geometry tests shall verify:

**topological validity**

**coordinate correctness**

**units**

**transformations**

**intersections**

**containment**

**offsets**

**joins**

**openings**

**assemblies**

**level alignment**

**element hosting**

and:

**geometric consistency across revisions**

---

## 19. PARAMETRIC TESTING

Parametric relationships shall be tested by changing controlling parameters and verifying dependent objects.

Example:

```
CHANGE LEVEL ELEVATION
↓
UPDATE HOSTED ELEMENTS
↓
```

UPDATE SPACES  
↓  
UPDATE VIEWS  
↓  
UPDATE DRAWINGS

The system shall verify that dependent objects remain coherent.

---

## 20. GEOMETRIC REGRESSION TESTING

Known geometry datasets shall produce known geometric signatures.

A geometry result shall be capable of producing:

geometry\_hash  
bounding\_box  
element\_count  
vertex\_count  
topology\_signature

Changes to expected geometry shall be detected automatically.

---

## 21. UNIT CONSISTENCY TESTING

OneDraft shall test conversions among supported units.

Initial units include:

mm  
cm  
m  
in  
ft

The system shall verify that conversion operations preserve dimensional meaning.

Architectural calculations shall never silently mix incompatible units.

---

## 22. CONSTRAINT TESTING

Constraint tests shall verify:

constraint creation  
constraint activation  
constraint evaluation  
constraint conflict  
constraint resolution

constraint preservation  
constraint invalidation

A model mutation that violates an active mandatory constraint shall fail validation or require an explicitly authorized workflow.

---

## 23. REQUIREMENT TRACEABILITY TESTING

Each mandatory project requirement shall be traceable to:

requirement  
↓  
affected object  
↓  
constraint or rule  
↓  
validation  
↓  
documentation  
↓  
acceptance

Tests shall detect requirements that have become disconnected from the computational model.

---

## 24. COMPLIANCE TESTING

The compliance subsystem shall be tested using controlled regulatory datasets.

Tests shall verify:

**code-source ingestion**

**rule extraction**

**rule applicability**

**jurisdiction matching**

**effective dates**

**code manifests**

**rule evaluation**

**exceptions**

**validation results**

## **provenance**

The test system shall distinguish test rules from authoritative production regulatory sources.

---

## **25. CODE MANIFEST TESTING**

A code manifest shall identify the exact regulatory inputs used for a validation.

Tests shall confirm:

project  
jurisdiction  
source  
version  
effective date  
content hash  
retrieval timestamp

remain associated with the validation operation.

---

## **26. VALIDATION ENGINE TESTING**

Validation tests shall cover:

PASS  
FAIL  
WARNING  
NOT\_APPLICABLE  
UNRESOLVED  
ERROR

Validation results shall identify:

rule  
affected object  
revision  
description  
severity  
source

---

## **27. VALIDATION DETERMINISM**

Where a validation operation is deterministic, identical inputs shall produce identical results.

Given:

Project

Revision  
Model Version  
Code Manifest  
Validation Configuration

the result should be reproducible.

Unexpected nondeterminism shall be treated as a defect.

---

## 28. DOCUMENTATION TESTING

Drawing generation shall be tested against known project models.

Tests shall verify:

**sheet creation**

**drawing placement**

**view generation**

**scale**

**dimensions**

**annotations**

**title blocks**

**sheet numbering**

**revision information**

**graphic standards**

**schedule generation**

---

## 29. DRAWING REGRESSION TESTING

Reference drawing sets shall be maintained for controlled test projects.

Generated drawings shall be compared against expected outputs.

Comparison mechanisms may include:

vector comparison  
geometry comparison  
metadata comparison  
rendered image comparison  
text extraction comparison  
sheet structure comparison

Pixel comparison alone shall not be considered sufficient.

---

## 30. DOCUMENT PACKAGE TESTING

Document generation tests shall verify:

- correct revision
- correct model version
- correct code manifest
- correct validation result
- correct drawing set
- correct sheet ordering
- correct filenames
- correct metadata
- correct hashes

The generated package shall be internally coherent.

---

## 31. REDLINE TESTING

Redline workflows shall be tested through:

```
OPEN
↓
INTERPRETED
↓
PROPOSED
↓
IMPLEMENTED
↓
VALIDATED
↓
RESOLVED
```

A redline shall not become RESOLVED merely because an AI response was generated.

Resolution requires implementation and verification.

---

## 32. REDLINE REGRESSION TESTING

Known redlines shall be associated with known expected model changes.

Example:

REDLINE:  
Move wall 300 mm north.

EXPECTED:

wall geometry changes  
dependent room geometry changes  
affected dimensions update  
affected drawings regenerate  
validation executes  
revision increments

The regression test shall verify the complete chain.

---

## 33. ARIA COMMAND TESTING

Aria commands shall be tested independently from the underlying AI model.

The testing boundary is:

ARIA OUTPUT  
↓  
COMMAND CONTRACT  
↓  
COMMAND VALIDATOR  
↓  
DOMAIN SERVICE

The system shall test whether the resulting command is valid rather than assuming the model output is correct.

---

## 34. AI COMMAND SAFETY TESTING

Tests shall reject commands that attempt:

direct SQL  
direct database writes  
permission changes  
billing changes  
unauthorized project access  
unsupported operations  
constraint bypass  
revision bypass  
production configuration changes

The AI layer shall remain inside the application capability boundary.

---

## 35. AI DETERMINISM TESTING

Where deterministic output is required, the system shall use controlled test conditions.



Where AI output is inherently probabilistic, testing shall evaluate:

schema validity  
command validity  
constraint compliance  
semantic correctness  
expected action  
unsafe behavior

rather than relying exclusively on exact text matching.

---

## 36. AI EVALUATION DATASETS

The platform shall maintain evaluation cases for:

**simple modifications**

**multi-object modifications**

**ambiguous instructions**

**conflicting requirements**

**constraint conflicts**

**invalid operations**

**unsupported requests**

**complex architectural changes**

**redline interpretation**

**documentation requests**

Evaluation datasets shall be version controlled.

---

## 37. END-TO-END TESTING

The principal OneDraft E2E test shall reproduce the complete project lifecycle:

```
CREATE ORGANIZATION
↓
CREATE USER
↓
CREATE PROJECT
↓
CREATE REQUIREMENTS
↓
CREATE SITE
↓
```

```
CREATE BUILDING
↓
CREATE LEVELS
↓
CREATE SPACES
↓
CREATE ELEMENTS
↓
CREATE CONSTRAINTS
↓
CREATE CODE MANIFEST
↓
VALIDATE
↓
GENERATE DRAWINGS
↓
CREATE REDLINE
↓
ARIA MODIFICATION
↓
CREATE REVISION
↓
REVALIDATE
↓
REGENERATE DRAWINGS
↓
GENERATE DOCUMENT PACKAGE
↓
ACCEPT
```

This becomes the primary system integration test.

---

## 38. REGRESSION TEST SUITE

Every production release shall execute the regression suite.

The regression suite shall include:

```
database migrations
API contracts
domain services
geometry
constraints
compliance
validation
documentation
AI commands
redlines
artifact generation
authentication
authorization
event processing
worker execution
```

A critical regression failure shall block release.

---

## 39. ACCEPTANCE TESTING

Acceptance tests verify that a completed feature satisfies its defined requirements.

Acceptance criteria shall be expressed in testable form.

Example:

```
GIVEN
a valid project with Revision 14

WHEN
a permitted user submits a valid wall movement command

THEN
the command is validated

AND
the model is modified

AND
a new revision is created

AND
affected drawings are identified

AND
validation is executed
```

---

## 40. TEST NAMING STANDARD

Tests shall use descriptive names.

Example:

```
test_move_element_preserves_required_setback
test_revision_rejects_stale_expected_revision
test_document_package_matches_model_version
test_redline_cannot_resolve_before_validation
test_code_manifest_is_immutable_after_finalization
```

Test names shall describe behavior rather than implementation.

---

## 41. TEST IDENTIFIERS

Every significant automated test should have a stable identifier.

Example:

TEST-MODEL-001  
TEST-GEOM-001  
TEST-CODE-001  
TEST-DOC-001  
TEST-ARIA-001  
TEST-E2E-001

Identifiers allow failures to be traced to requirements and specifications.

---

## 42. REQUIREMENT-TO-TEST TRACEABILITY

Requirements shall map to verification tests.

The relationship becomes:

REQUIREMENT  
↓  
IMPLEMENTATION  
↓  
TEST  
↓  
RESULT  
↓  
RELEASE

Critical requirements without verification coverage shall be reported.

---

## 43. TEST RESULT MODEL

Test executions shall produce structured records containing:

test\_id  
test\_suite  
environment  
commit\_hash  
build\_id  
started\_at  
completed\_at  
status  
duration  
input\_reference  
output\_reference  
failure\_details

---

## 44. TEST STATUS

Initial statuses:

QUEUED  
RUNNING  
PASSED  
FAILED  
SKIPPED  
BLOCKED  
CANCELLED

A skipped test must include a reason.

---

## 45. FAILURE CLASSIFICATION

Failures shall be classified as:

APPLICATION  
DATABASE  
API  
GEOMETRY  
COMPLIANCE  
VALIDATION  
DOCUMENTATION  
AI  
INFRASTRUCTURE  
SECURITY  
CONFIGURATION  
DATA  
TEST\_HARNESS

This supports automated triage.

---

## 46. QUALITY GATES

A release shall not proceed when required quality gates fail.

Initial gates:

BUILD SUCCESS  
UNIT TEST SUCCESS  
DOMAIN TEST SUCCESS  
DATABASE TEST SUCCESS  
API CONTRACT SUCCESS  
INTEGRATION TEST SUCCESS  
REGRESSION SUCCESS  
SECURITY TEST SUCCESS

Additional gates may be introduced as the platform matures.

---

## 47. CODE COVERAGE

Code coverage shall be measured continuously.

Coverage shall include:

- line coverage
- branch coverage
- function coverage

Coverage percentages shall not be treated as the sole measure of software quality.

Critical domain behavior shall receive explicit tests even where aggregate coverage is high.

---

## 48. GEOMETRY COVERAGE

Geometry testing shall measure behavioral coverage across:

- element types
- topological conditions
- assemblies
- intersections
- openings
- host relationships
- levels
- transforms
- constraints

A high software-code coverage percentage does not constitute adequate geometry coverage.

---

## 49. API COVERAGE

The testing system shall track coverage of:

- endpoints
- methods
- authentication paths
- authorization paths
- success responses
- error responses
- validation responses
- concurrency conflicts

---

## 50. DATABASE COVERAGE

Database verification shall cover:

- tables
- constraints
- relationships
- transactions
- migrations
- indexes
- historical records
- soft deletion

---

## 51. PERFORMANCE TESTING

Performance testing shall measure:

- API latency
- database latency
- geometry computation time
- validation time
- drawing generation time
- document generation time
- queue latency
- worker throughput

Performance targets shall be defined by workload class.

---

## 52. LOAD TESTING

Representative workloads shall include:

- single-user project
- small project
- medium project
- large project
- large drawing package
- many concurrent projects
- many concurrent workers
- high-frequency API requests

The system shall identify capacity limits before production deployment.

---

## 53. STRESS TESTING

Stress testing shall intentionally exceed normal operating conditions.

The purpose is to identify:

failure thresholds  
resource exhaustion  
queue saturation  
database contention  
memory pressure  
worker instability  
recovery behavior

---

## 54. SECURITY TESTING

Security tests shall verify:

**authentication**

**authorization**

**tenant isolation**

**input validation**

**SQL injection resistance**

**object-level authorization**

**token handling**

**secret handling**

**file access controls**

**API abuse controls**

**audit logging**

Security testing shall operate within authorized test environments.

---

## 55. MULTI-TENANT TESTING

The system shall explicitly test that:

Organization A

cannot access:

Organization B

resources.



Tests shall cover:

projects  
models  
drawings  
documents  
artifacts  
jobs  
events  
billing data

Tenant isolation failures shall be release-blocking.

---

## 56. ARTIFACT TESTING

Generated artifacts shall be tested for:

existence  
readability  
content type  
size  
hash  
metadata  
revision  
model version  
project association

Artifacts shall not become authoritative merely because a file was successfully generated.

---

## 57. HASH VERIFICATION

Where artifacts use content hashing, tests shall verify:

generated artifact  
↓  
hash  
↓  
stored hash

matches exactly.

A mismatch shall invalidate the artifact record.

---

## 58. EVENT TESTING

The Integration & Event Bus shall be tested for:

- event publication
- event consumption
- schema validity
- ordering where required
- idempotency
- duplicate delivery
- retry behavior
- dead-letter handling

---

## 59. WORKER TESTING

Workers shall be tested independently and through integration workflows.

Tests shall verify:

- job acquisition
- job execution
- progress reporting
- failure handling
- retry
- cancellation
- timeout
- artifact creation
- completion event

---

## 60. RETRY TESTING

Retryable operations shall be tested against:

- temporary database failure
- temporary service failure
- worker interruption
- network interruption
- artifact-storage interruption
- event delivery failure

Retries shall not produce duplicate project mutations.

---

## 61. CANCELLATION TESTING

Long-running jobs shall support controlled cancellation where appropriate.

Tests shall verify:

- QUEUED → CANCELLED
- RUNNING → CANCELLED

and shall ensure that cancellation does not leave corrupt or falsely completed artifacts.

---

## 62. RECOVERY TESTING

The testing system shall verify recovery after:

worker crash  
API restart  
database restart  
queue interruption  
object-storage interruption  
network failure

Recovery must preserve authoritative project state.

---

## 63. REPRODUCIBILITY TEST

A core OneDraft test shall verify that a project can be reconstructed from:

project\_id  
revision\_id  
model\_version\_id  
code\_manifest\_id

The system shall regenerate the associated computational and documentation state.

Failure indicates a provenance defect.

---

## 64. HISTORICAL RECONSTRUCTION TEST

Given an earlier revision:

Revision 10

the system must be able to reconstruct the corresponding model state without replacing the current:

Revision 15

Historical state must remain independently recoverable.

---

## 65. DOCUMENT PROVENANCE TEST

A final document package shall be traceable to:

Project  
Model Version  
Revision  
Code Manifest  
Validation Run  
Drawings  
Generation Job  
Artifact Hash  
Acceptance

Tests shall verify that no link in this chain is missing.

---

## 66. RELEASE CANDIDATE TESTING

Before release, the system shall create a release candidate identified by:

version  
commit\_hash  
build\_id  
environment  
timestamp

The release candidate shall execute the complete mandatory test suite.

---

## 67. RELEASE VERIFICATION RECORD

Each release shall produce a verification record containing:

release\_version  
commit\_hash  
build\_id  
test\_suite\_version  
passed\_tests  
failed\_tests  
skipped\_tests  
security\_status  
migration\_status  
deployment\_status  
verified\_at

---

## 68. TEST REPORTING

The platform shall provide machine-readable test results.

Preferred formats include:

JSON  
JUnit XML

Human-readable reports may additionally be generated as:

HTML  
PDF

---

## 69. CONTINUOUS INTEGRATION

Every code change shall trigger appropriate automated testing.

The minimum CI sequence shall be:

```
CHECKOUT
↓
DEPENDENCY INSTALL
↓
STATIC ANALYSIS
↓
UNIT TESTS
↓
DOMAIN TESTS
↓
DATABASE TESTS
↓
API TESTS
↓
INTEGRATION TESTS
```

Additional suites shall run according to change scope.

---

## 70. CHANGE-BASED TEST SELECTION

The CI system may optimize execution by identifying affected subsystems.

For example:

```
geometry change
↓
geometry tests
↓
model tests
↓
```

validation tests

↓

documentation regression tests

However, mandatory release validation shall still execute the complete required regression suite.

---

## 71. TEST ARTIFACT STORAGE

Test artifacts shall be stored separately from production project artifacts.

Test artifacts may include:

- logs
- coverage reports
- screenshots
- reference drawings
- geometry snapshots
- database dumps
- test packages
- failure traces
- performance results

Each artifact shall identify its test execution.

---

## 72. TEST ENVIRONMENT VERSIONING

Every test execution shall record:

- application version
- database migration version
- container image versions
- dependency versions
- configuration version
- test dataset version

This ensures failures can be reproduced.

---

## 73. FLAKY TEST MANAGEMENT

Tests exhibiting nondeterministic behavior shall be identified as:

FLAKY

Flaky tests shall not be silently ignored.

Each flaky test requires:

owner  
reason  
tracking reference  
remediation plan

---

## 74. TEST QUARANTINE

A temporarily unstable test may be quarantined only under controlled conditions.

Quarantine shall not be used to bypass critical verification.

The system shall record:

test\_id  
reason  
date  
owner  
approval  
expiration

---

## 75. DEFECT MANAGEMENT

Test failures shall create structured defects where appropriate.

A defect should identify:

test\_id  
environment  
build\_id  
project\_id  
revision\_id  
error  
expected behavior  
actual behavior  
reproduction information

---

## 76. QUALITY DASHBOARD

The platform should expose operational quality metrics including:

test pass rate  
failure rate  
coverage  
flaky test count  
regression failures  
build failures  
API failures

geometry failures  
validation failures  
document failures  
security findings  
performance trends

---

## 77. TESTING OF TESTS

Critical test infrastructure shall itself be tested.

The system shall periodically introduce controlled faults or mutation cases to verify that tests actually detect defects.

Examples:

incorrect constraint  
invalid revision  
broken relationship  
incorrect geometry  
missing drawing  
incorrect code manifest  
unauthorized access

---

## 78. MUTATION TESTING

Selected domain components may use mutation testing.

The objective is to verify that tests fail when expected behavior is intentionally altered.

Mutation targets should prioritize:

constraints  
authorization  
revision logic  
geometry calculations  
validation logic  
command safety

---

## 79. CONTRACT COMPATIBILITY

API changes shall be checked for compatibility with:

frontend  
workers  
generated clients  
external integrations



SDKs

Breaking changes require an API version transition.

---

## 80. BACKWARD COMPATIBILITY

Where supported, OneDraft shall test compatibility with:

- previous project revisions
- previous document packages
- previous API clients
- previous event schemas

Migration logic shall be verified before production deployment.

---

## 81. SCHEMA COMPATIBILITY

Database schema changes shall be evaluated for:

- existing data
- historical revisions
- running workers
- API compatibility
- migration safety
- rollback/recovery

Destructive schema changes require explicit migration procedures.

---

## 82. ACCEPTANCE REGRESSION

Once a project has passed customer acceptance, its immutable acceptance record shall remain verifiable.

The system shall be able to prove:

- accepted revision
- accepted model version
- accepted document package
- accepted code manifest

remain unchanged.

---

## 83. PROFESSIONAL REVIEW TESTING

Where professional review records exist, the system shall verify:

reviewer identity  
review scope  
reviewed revision  
review result  
review timestamp

Professional review records shall remain associated with the exact project state reviewed.

---

## 84. FINALIZATION TEST

A finalized project package shall satisfy all required gates:

MODEL VALID  
↓  
CONSTRAINTS VALID  
↓  
COMPLIANCE VALID  
↓  
DOCUMENTATION VALID  
↓  
PACKAGE VALID  
↓  
PROVENANCE VALID  
↓  
ACCEPTANCE

A package shall not be marked final merely because generation completed successfully.

---

## 85. TESTING STATE MACHINE

The testing subsystem shall use:

DEFINED  
↓  
QUEUED  
↓  
RUNNING  
↓  
PASSED / FAILED  
↓  
REVIEWED  
↓  
ACCEPTED

Release-blocking failures shall prevent progression.

---

## 86. TEST COMMAND API

The API shall eventually expose controlled testing operations.

Conceptually:

```
POST /api/v1/tests/runs
GET  /api/v1/tests/runs/{test_run_id}
GET  /api/v1/tests/runs/{test_run_id}/results
GET  /api/v1/tests/suites
```

Administrative access shall be required for execution of privileged test operations.

---

## 87. TEST RUN REQUEST

Conceptually:

```
{
  "suite": "E2E_PROJECT_LIFECYCLE",
  "environment": "TEST",
  "dataset_id": "uuid",
  "configuration": {}
}
```

Response:

```
{
  "data": {
    "test_run_id": "uuid",
    "status": "QUEUED"
  }
}
```

---

## 88. TEST RESULT RESPONSE

Conceptually:

```
{
  "data": {
    "test_id": "TEST-E2E-001",
    "status": "PASSED",
    "duration_ms": 8421,
    "artifacts": [
      "uuid"
    ]
  }
}
```

---

## 89. QUALITY GATE RESPONSE

A release verification endpoint may return:

```
{
  "data": {
    "release": "0.1.0",
    "status": "BLOCKED",
    "failed_gates": [
      "GEOMETRY_REGRESSION"
    ]
  }
}
```

The deployment system shall respect this result.

---

## 90. TEST DATABASE

A dedicated PostgreSQL test database shall be used for integration testing.

Test databases may be created dynamically per test run where practical.

Example:

onedraft\_test\_<run\_id>

The database shall be disposable.

---

## 91. TEST ISOLATION

Tests shall not depend on execution order unless explicitly designed as workflow tests.

Unit and integration tests shall clean up or recreate their state.

End-to-end tests may intentionally preserve their complete execution chain for reconstruction verification.

---

## 92. TEST TIME CONTROL

Tests involving:

timestamps  
jobs

timeouts  
scheduled operations  
retention  
expiration

shall support controlled test time where practical.

This prevents tests from depending on actual wall-clock timing.

---

## 93. EXTERNAL SERVICE MOCKING

External services may be mocked for deterministic tests.

Examples:

AI provider  
email  
payment provider  
external CAD services  
external regulatory services  
object storage  
event delivery

Production integration tests shall additionally use controlled real-service environments where required.

---

## 94. AI PROVIDER TEST DOUBLES

The Aria provider abstraction shall permit deterministic test implementations.

Example:

MockAriaProvider  
ReplayAriaProvider  
EvaluationAriaProvider  
ProductionAriaProvider

The application should not require a live AI provider to execute core domain tests.

---

## 95. REPLAY TESTING

Recorded command scenarios may be replayed against new software versions.

Example:

Input Project State  
+  
Aria Command

+  
Expected Result

The system executes the command against the current implementation and compares the result.

---

## 96. GOLDEN DATASETS

Golden datasets shall represent known-good project states.

Each dataset shall include:

project model  
geometry  
constraints  
code manifest  
validation results  
drawings  
document package  
hashes

Changes to golden datasets require explicit review.

---

## 97. GOLDEN DOCUMENT TESTING

Known-good drawing and document packages shall be used to detect unintended output changes.

Changes shall be classified as:

EXPECTED  
UNEXPECTED  
REVIEW\_REQUIRED

---

## 98. TESTING ARCHITECTURAL PROVENANCE

The test system shall verify the complete OneDraft provenance chain:

USER REQUIREMENT  
↓  
PROJECT MODEL  
↓  
GEOMETRY  
↓  
COMPLIANCE  
↓

VALIDATION

↓

DOCUMENTATION

↓

ARTIFACT

↓

ACCEPTANCE

Every transition must remain attributable.

---

## 99. MVP TESTING SUCCESS CRITERIA

The testing system is successful when OneDraft can automatically verify:

**database migrations**

**API contracts**

**project creation**

**model creation**

**geometry operations**

**constraints**

**revisions**

**Aria commands**

**compliance manifests**

**validation**

**drawing generation**

**redlines**

**document packages**

**artifact hashes**

**authorization**

**worker execution**

**event delivery**

**customer acceptance**

and:

**historical reconstruction**

---

# 100. FIRST COMPLETE QUALITY TEST

The first complete OneDraft quality test shall execute:

```
CREATE ORGANIZATION
↓
CREATE USER
↓
CREATE PROJECT
↓
CREATE REQUIREMENTS
↓
CREATE SITE
↓
CREATE BUILDING
↓
CREATE LEVELS
↓
CREATE ROOMS
↓
CREATE WALLS
↓
CREATE DOORS
↓
CREATE WINDOWS
↓
CREATE CONSTRAINTS
↓
CREATE CODE MANIFEST
↓
RUN VALIDATION
↓
GENERATE PLANS
↓
GENERATE ELEVATIONS
↓
GENERATE SECTION
↓
GENERATE ARCH D SHEETS
↓
SUBMIT ARIA COMMAND
↓
VERIFY COMMAND
↓
CREATE REVISION
↓
REGENERATE AFFECTED DRAWINGS
↓
CREATE REDLINE
↓
RESOLVE REDLINE
↓
RUN REGRESSION VALIDATION
↓
GENERATE DOCUMENT PACKAGE
↓
HASH PACKAGE
↓
```



VERIFY PROVENANCE

↓

RECORD ACCEPTANCE

↓

RECONSTRUCT PROJECT HISTORY

This becomes the first comprehensive OneDraft quality-assurance regression workflow.

---

## 101. VERSION 0.1 ENGINEERING STATE

The OneDraft architecture now contains a dedicated verification layer covering:

**software testing**

**domain verification**

**database verification**

**API contract testing**

**geometry testing**

**parametric testing**

**compliance testing**

**validation testing**

**documentation testing**

**AI command testing**

**redline testing**

**worker testing**

**event testing**

**security testing**

**performance testing**

**regression testing**

**end-to-end testing**

**acceptance testing**

**provenance verification**

---

# 102. NEXT IMPLEMENTATION ARTIFACTS

The next engineering package derived from this specification shall consist of:

## **Artifact A**

tests/unit/

Core unit and domain tests.

## **Artifact B**

tests/integration/

Database, API, service, worker, event, and artifact integration tests.

## **Artifact C**

tests/geometry/

Parametric and geometric regression tests.

## **Artifact D**

tests/compliance/

Regulatory-rule and validation verification datasets.

## **Artifact E**

tests/e2e/

Complete project lifecycle test.

## **Artifact F**

tests/fixtures/

Version-controlled project and model fixtures.

## **Artifact G**

tests/golden/

Known-good geometry, drawing, and document datasets.

## **Artifact H**

tests/contracts/

OpenAPI and service-contract verification.

## Artifact I

tests/security/

Authorization, isolation, input validation, and security regression tests.

## Artifact J

tests/performance/

Load, stress, concurrency, geometry, validation, and documentation benchmarks.

---

# 103. FINAL ARCHITECTURAL DECISION

OneDraft shall treat automated verification as part of the computational architecture rather than as an external development convenience.

The system must continuously answer:

Did the software execute correctly?

Did the domain model remain valid?

Did the geometry remain valid?

Were constraints preserved?

Was the applicable regulatory state preserved?

Was validation performed?

Were the drawings generated from the correct revision?

Was the document package generated from the correct model?

Was the artifact preserved?

Can the result be reconstructed?

Can the result be independently verified?

The answer to each question must be represented through executable tests, structured records, or deterministic verification mechanisms.

Testing therefore becomes part of the **OneDraft system of evidence**.

---

# 104. SPECIFICATION STATUS

ONEDRAFT TESTING, QUALITY ASSURANCE & VERIFICATION SYSTEM v0.1

## **STATUS: ESTABLISHED**

The next implementation stage is the creation of the executable testing foundation:

tests/unit/

tests/integration/

tests/geometry/

tests/compliance/

tests/e2e/

tests/fixtures/

tests/golden/

tests/contracts/

tests/security/

tests/performance/

These artifacts establish the automated verification layer for the OneDraft computational platform.